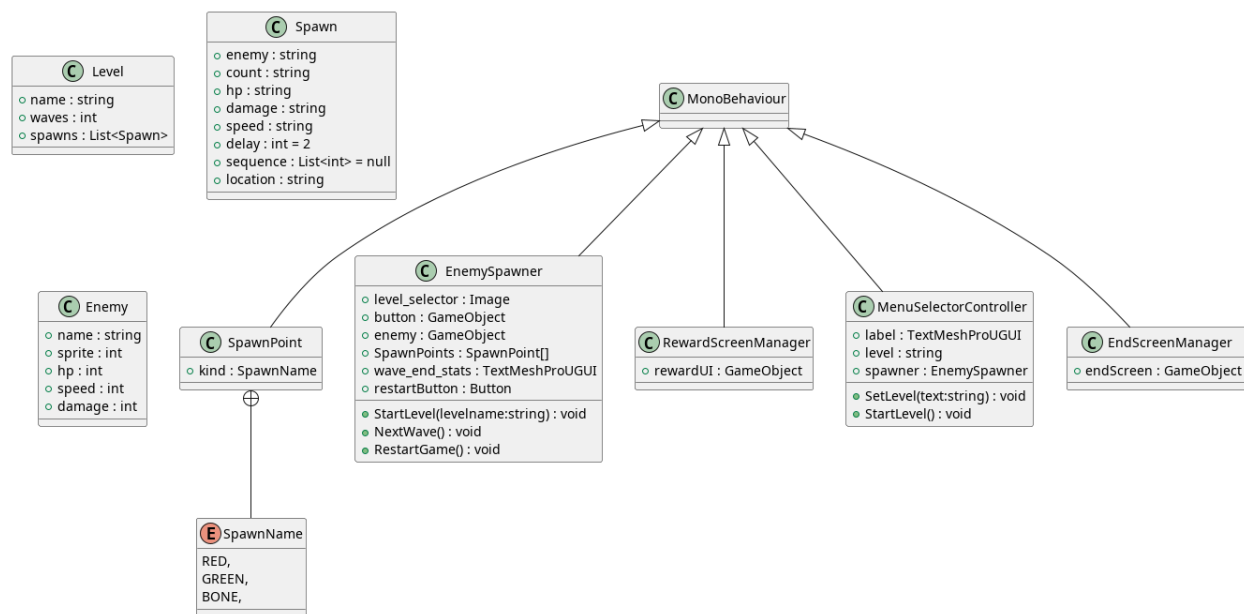


# CMPM 121 Report.pdf - Claire Buck and Calvin Richards

## Architecture Diagram



## Textual Description of Architecture

The overall architecture of this project is data-driven, meaning that level difficulty and enemies and their spawning are all very flexible and easy to change without having to change the code itself. This structure allows new enemies or levels to be added to the JSON file of their information, and then nothing will have to be edited in the other files in order for that new information to work.

GameManager is the backbone of the game: it tracks global game states (PREGAME, INWAVE, etc.), enemy lists, and the player's stats. EnemySpawner is another key file, and it reads level and enemy data from JSON files, supports RPN expressions, and controls game setup, enemy spawning and game resetting. Combat between the player and enemies is contained mostly in the Hittable class, which is used in both EnemyController and PlayerController, both of which contain information for their respective types of sprites and have damage and death events.

## Personal Paragraphs

### Calvin's Paragraph

I created the data object classes ([Enemy.cs](#), [Level.cs](#), [Spawn.cs](#)) and wrote the entirety of the SpawnEnemy() function. I implemented the JSON deserialization code (after searching up how to deserialize JSON in C#) and used it to populate the aforementioned data objects. I also wrote the SpawnWave() function and followed the professor's advice by separating the spawning of different enemy waves between SpawnEnemy() and SpawnEnemies(). Claire and I split the work on SpawnEnemies(). I

also created the Unity GameObject EndManager and its corresponding Canvas item EndScreen, along with its script. I felt somewhat prepared for this assignment due to contributing probably 95% of the work for my recent assignment in CMPM 125, which led me to expect to be searching through Unity objects in the inspector to find their corresponding scripts, as well as combing through each script to understand the overall control flow of the game. Reading through the entirety of the instructions multiple times was also very helpful in that process. On top of this, I have also created various basic game engines in my own time, so implementing state machine style behavior in the various objects of the game came naturally to me. That being said, there were some very confusing things about working in this framework – the EnemySpawner being owned by the Main Camera which is in turn owned by the Player was quite confusing and took a while to find. Understanding that the Canvas items controlled the presence of the on-screen buttons took a while, prior to that we were both very confused why the buttons we were adding in code were not showing up. Overall I found the project manageable, but it required a lot of reading and re-reading to comprehend the pre-existing structure of the framework before being able to implement the necessary features.

### Claire's Paragraph

I contributed my RPN calculator file, the new enemy type, incorporating our new enemy type into waves, the level selector buttons, the text that displays information between waves, and the stuff for restarting the game after a loss or win. I think the main thing I struggled with this project was just trying to understand what all the different files did and how they worked together, especially within the framework of Unity. I have never used Unity before, and while I didn't find it overly challenging, I didn't dive too deep into it and I also had to get help from Calvin about how to use certain aspects for it. For example, I was trying to just get the restart button to show up for the longest time, and Calvin showed me how to add it through the Unity editor. It was also tricky to figure out how to totally reset the game when it restarts, because I had to remember all the things that needed to be reset or remade, and then also figure out the best way to do each one. Going forward, I feel like I definitely need to become more familiar with Unity for the next project, and spend more time poring over the files in order to understand what they all do. I think the thing that slowed me the most in this project was finding where all the files connected to each other, what they do, and where certain things are stored and how to access them.